

Midterm Project Report

Team Member: Qistina Zamzuri

Course: CMPSC448 Machine Learning and Algorithmic AI

Instructor: Prof. Wenpeng Yin

Teaching Assistant: Saatvik Pradhan

Submission Date: Oct 11th , 2026

Table of Contents

Midterm Project Report *1*

Identifying the Source LLM of a Response with CNN and LSTM Classifiers *3*

Abstract **3**

1. Problem and dataset curation **3**

Research questions 4

Source 4

Curation steps (prepare.py) 4

Self-identification masking 4

2. Models and training **5**

Input representation 5

TextCNN 5

BiLSTM 5

Baselines 5

Training setup 5

3. Results **7**

4. Analysis **9**

RQ1: can CNNs and LSTMs identify the source LLM? 9

RQ2: prompt, response or both? 10

Does self-identification matter? ..... **Error! Bookmark not defined.**

5. Lessons learned **11**

Limitations ..... **Error! Bookmark not defined.**

Future work ..... **Error! Bookmark not defined.**

Repository and reproducibility ..... Error! Bookmark not defined.

AI-use statement ..... Error! Bookmark not defined.

References ..... Error! Bookmark not defined.

Midterm Project Report: Identifying the Source LLM of a Response with CNN and LSTM Classifiers

Abstract

I trained a TextCNN and a bidirectional LSTM to classify responses from the Chatbot Arena dataset into five LLM families: Claude, Gemini, GPT, Llama and Mistral. Using the response alone, the LSTM reaches a macro-F1 of 0.615 ± 0.007 and the CNN 0.610 ± 0.003 (mean ± std over 3 seeds), about three times the chance level of 0.200. A TF-IDF + logistic regression baseline does as well (0.617). The user's prompt carries no usable signal: prompt-only models score at chance (0.185–0.207), and adding the prompt to the response does not help (0.595–0.603). Claude is the easiest family to recognize (F1 ≈ 0.85) and Mistral the hardest (F1 ≈ 0.40–0.45), being confused mostly with GPT and Llama. And masking the models' self-identifying names ("OpenAI", "Claude", and so on) changes macro-F1 by less than one standard deviation, so the classifiers rely on style rather than self-mentions. The CNN still generalizes to it despite was trained without a task, losing about 0.06 F1 on coding and math and nothing on writing.

Problem and dataset curation

Research questions

Each example is a triple (LLM name, input, output): a user prompt and the response one LLM gave to it. I study three questions:

1. RQ1**:** Can a CNN and an LSTM identify which LLM family generated a response?
2. RQ2**:** Which input works best: the prompt only, the response only, or both?
3. RQ3 (extra): Does a classifier trained on some tasks generalize to a task it never saw?

Source

I use the publicly released Chatbot Arena human-preference dataset, lmarena-ai/arena-human-preference-55k (Apache-2.0; Chiang et al., 2024). It contains about 57,000 "battles" in which a real user wrote a prompt and two anonymous LLMs answered it. Each battle therefore gives two labelled examples that share one prompt. The data was collected and released by LMArena; I collected no personal data myself.

Curation steps (`prepare.py`)

1. First turn only: Many conversations have several turns. I keep the first user prompt and each model's first response, so every example answers one self-contained prompt.
2. Model families: Model names are mapped to families by prefix: gpt-3.5*/gpt-4* → GPT, claude* → Claude, llama* → Llama, mistral*/mixtral* → Mistral, gemini* → Gemini. Fine-tuned models trained on other models' outputs (Vicuna, Koala, WizardLM and similar) are dropped because their style would blur the family boundaries.
3. Cleaning: Empty responses and exact duplicate (family, prompt, response) rows are removed.
4. Balancing: Every family is downsampled to the same size, [N = 2869] examples per family ([5N = 14345] in total), so accuracy and macro-F1 are not driven by class frequency and chance is exactly 1/5 = 0.200.
5. Task tags: Each prompt gets a rough task label (coding, math, writing or other) from keyword rules, for the cross-task experiment (RQ3).
6. Prompt-grouped split: Train/validation/test = 70/15/15, made with GroupShuffleSplit grouped on the lowercased prompt text. All examples that share a prompt land in the same split, so a test prompt is never seen during training (see Section 5). The test set has 2,159 examples: Claude 437, Gemini 420, GPT 422, Llama 434, Mistral 446.

Self-identification masking

Models sometimes name themselves ("As an AI developed by OpenAI…")and such mentions would let a classifier read the label straight off the text. Hence, before training, I replace model and company names (OpenAI, ChatGPT, GPT-4, Anthropic, Claude, Meta AI, Llama, Google, Gemini, Bard, Mistral, Qwen and others) with an `<org>` token. A `--nomask` flag keeps them, for an ablation.

2. Models and training

Input representation

Formatting is a large part of an LLM's style, so the tokenizer keeps it: case is preserved, punctuation (including markdown symbols such as *, # and -) becomes separate tokens, and every line break becomes a <nl> token. The vocabulary holds the 30,000 most frequent training tokens that appear at least twice, plus <pad>, <unk> and <sep>. Sequences are also cut to 256 tokens. The three input modes are:

1. prompt**:** the first 256 prompt tokens
2. response**:** the first 256 response tokens
3. both**:** 64 prompt tokens, <sep>, then up to 191 response tokens

TextCNN

Following Kim (2014): a 200-dimensional embedding, then parallel 1-D convolutions of width 3, 4 and 5 with 128 filters each and ReLU, max-pooling over time, concatenation (384 features), dropout 0.5 and a linear layer to the five classes. Padding positions are masked out before pooling, so a short response is not padded with zeros that the max could pick up. About 6.31 million parameters, 96% of them in the embedding.

BiLSTM

A 200-dimensional embedding with dropout 0.5, then a 2-layer bidirectional LSTM with 128 hidden units per direction (inter-layer dropout 0.5), run on packed sequences so I skip padding. The outputs are max-pooled over time (padding masked), followed by dropout 0.5 and a linear layer. There are about 6.73 million parameters.

Baselines

1. Majority class**:** always predicts the most frequent training class.
2. TF-IDF + logistic regression**:** word unigrams and bigrams from the same tokenizer (min document frequency 2, up to 100,000 features, sublinear TF), with logistic regression (C = 4).

Training setup

Setting	Value
Optimizer	Adam, learning rate 1e-3
Loss	Cross-entropy
Batch size	64
Max epochs	15
Early stopping	On validation macro-F1, patience 3; the best checkpoint is restored

Setting	Value
Gradient clipping	Norm 1.0
Seeds	3 (1, 2, 3); report mean ± standard deviation
Hardware	Google Colab, NVIDIA T4 GPU
Metrics	Accuracy and macro-F1 on the held-out test set

I report macro-F1 as the main metric because it weights every family equally. Since the classes are balanced, accuracy and macro-F1 are close throughout.

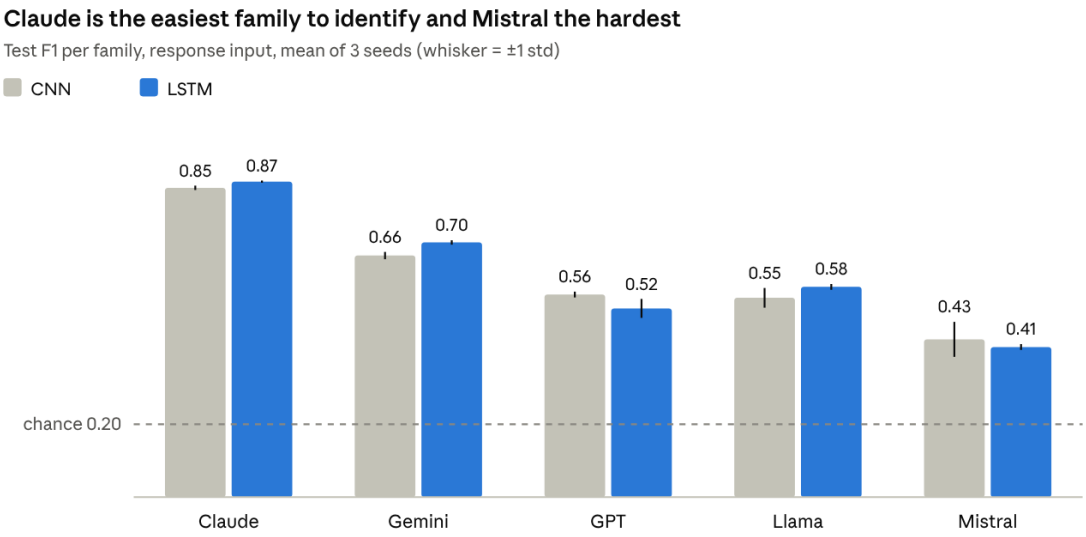
3. Results

Responses alone identify the LLM family well above chance; prompts alone do not. Table 1 lists test-set results for every model and input mode, all with self-identification masked. Neural rows are the mean ± standard deviation over 3 seeds; baselines are deterministic.

Table 1. Test accuracy and macro-F1 (5 balanced classes, chance = 0.200)

Model	Input	Accuracy	Macro-F1
Majority class	–	0.195	0.065
TF-IDF + LR	prompt	0.207	0.207
TF-IDF + LR	response	0.616	**0.617**
TF-IDF + LR	both	0.602	0.602
TextCNN	prompt	0.198 ± 0.006	0.189 ± 0.013
TextCNN	response	0.609 ± 0.003	0.610 ± 0.003
TextCNN	both	0.601 ± 0.005	0.603 ± 0.001
BiLSTM	prompt	0.200 ± 0.004	0.185 ± 0.002
BiLSTM	response	**0.617 ± 0.007**	**0.615 ± 0.007**
BiLSTM	both	0.587 ± 0.014	0.595 ± 0.014

With the response as input, the BiLSTM, the TextCNN and TF-IDF are within about one standard deviation of each other (0.610–0.617), so neither neural model beats the linear baseline. Accuracy per family is far from even, as the figure below shows:



results/<model>-response/seed1-3.json · test set, 2,159 examples

The CNN's best epoch ranged from 6 to 8. The LSTM's ranged from 6 to 15, and two of its three response runs were still improving at the 15-epoch limit, so the LSTM may gain a little from longer training.

4. Analysis

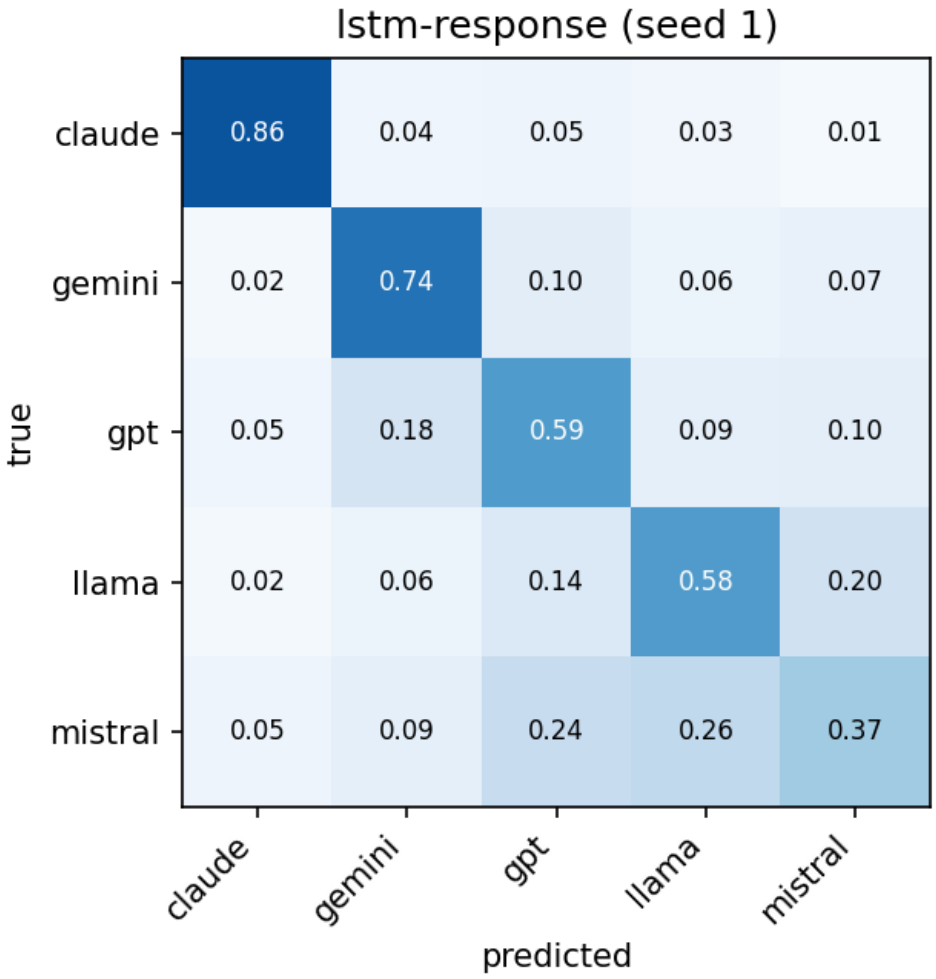
RQ1: can CNNs and LSTMs identify the source LLM?

Yes, moderately well. From 256 tokens of a response, both models pick the correct family out of five about 61% of the time, three times the 20% chance rate. But they do no better than TF-IDF + logistic regression (0.617), a bag of word unigrams and bigrams. That suggests the signal sits mostly in which words and formatting tokens appear, not in longer-range word order that convolutions or recurrence could exploit.

The largest TF-IDF weights (`results/tfidf-top-features.txt`) show what those signals are:

Family	Highest-weight features
Claude	"Here is", "So", "in summary", "I apologize", bullet "•"
Gemini	bold markdown `* *`, list items `<nl> *`, "realm", "It is"
GPT	"Remember", "crucial", "Please note", "As an", "m sorry", "assistance"
Llama	"Sure !", "!", "Sure ,", "just an", "such as"
Mistral	"Note :", "However", an escaped asterisk `\*`, "always"

These are recognizable habits: Llama's cheerful "Sure! Here's…" opening, GPT's "Remember…" and "It's crucial…", Claude's "Here is…" and apologies, and Gemini's heavy bold markdown. "As an" still points to GPT after masking because the mask removes names ("As an AI developed by `<org>`"), not the phrase itself.



The confusion matrix (BiLSTM, response input, seed 1) shows where the errors go. Claude is recognized 86% of the time and rarely confused with anything. Mistral is the weak class: only 37% of Mistral responses are labelled Mistral, while 26% go to Llama and 24% to GPT. Llama is in turn mistaken for Mistral 20% of the time, and GPT for Gemini 18% of the time. One plausible reason is that our Mistral class mixes several quite different models (Mistral-7B, Mistral-Medium and Mixtral), and open-weight instruction-tuned models may share fine-tuning data with Llama-style models. But, I did not test this hypothesis.

RQ2: prompt, response or both?

The prompt alone carries no signal. All three prompt-only models score at chance (macro-F1 0.185–0.207). This fits how Chatbot Arena works: the two models in a battle are assigned to the user's prompt at random, so the prompt says nothing about which model answered. I had expected a small above-chance effect from time drift (models active in different months see different prompt topics), but none appeared.

Furthermore, adding the prompt to the response does not help. "Both" scored 0.603 (CNN) and 0.595 (LSTM), slightly below response-only. With a fixed 256-token budget, the 64 prompt

tokens push out response tokens, leaving at most 191. The models therefore see less of the only informative text in exchange for uninformative text. The LSTM in this mode also had the highest seed-to-seed variance (±0.014).

Answer to RQ2**:** use the output only.

RQ3: does the classifier generalize to an unseen task?

Using the task tags from Section 1, rq3.py compares two TextCNNs (response input, masked, 3 seeds) on the test rows of each task: one trained on all tasks (seen), and one trained on every task except that one (unseen). Both use the same architecture and settings as Section 2.

Table 2. Macro-F1 on each task's test rows, seen vs. unseen during training (mean ± std over 3 seeds)

Held-out task	Test rows	Seen	Unseen	Drop
Coding	249	0.653 ± 0.030	0.588 ± 0.028	0.064
Math	169	0.597 ± 0.014	0.542 ± 0.026	0.055
Writing	265	0.578 ± 0.029	0.577 ± 0.017	0.000

The classifier mostly transfers, but less well to technical tasks. All unseen scores stay far above chance (0.200), so much of a family's fingerprint is task-independent. Coding and math lose about 0.06 F1, roughly two standard deviations, when the model never saw that task. Writing loses nothing. A plausible reading is that coding and math answers carry task-specific habits, such as how each family formats code blocks, equations and step-by-step solutions, which the model only learns from in-task examples. Creative and explanatory writing, by contrast, is close to the general prose the other tasks already contain. The test sets are small (169–265 rows) and the seed spread is 0.014–0.030, so we read the coding and math drops as moderate evidence, and the writing result as "no detectable drop" rather than proof of perfect transfer.

Does self-identification matter?

I also trained the CNN on responses with model and company names left in (--nomask). Macro-F1 rose from 0.610 ± 0.003 to 0.615 ± 0.009, a difference smaller than one standard deviation. Explicit self-mentions are therefore rare enough that the classifiers are learning style, not reading a name tag. Mistral gained the most (F1 0.433 → 0.468), which may mean Mistral models name themselves more often; hence, this would need checking on more seeds.

5. Lessons learned

1. Better to split by the shared unit, not by row. My first version split rows at random. In Arena both answers in a battle share one prompt, so the same prompt could appear in train and test. I switched to a split grouped by prompt. The response-only TF-IDF score barely changed, because a shared prompt's two answers come from different families, but the grouped split is what makes the prompt-only and prompt + response results trustworthy.
2. Run a strong simple baseline first as TF-IDF + logistic regression trains in seconds and matched both neural models. Without it, I might have credited the CNN and LSTM with learning something a bag of n-grams already captures.
3. Preprocessing choices are modelling choices. I learnt this when I realised lowercasing and stripping punctuation, the usual defaults, would have removed many of the strongest cues: markdown bullets and bold, "!", and line breaks.
4. Validation loss and validation F1 can disagree. In the LSTM runs, validation loss started rising after about epoch 5 while validation macro-F1 kept creeping up (see results/lstm-response/curves-seed1.png): the model grew more confident on its mistakes while still getting more answers right. Early stopping on macro-F1 rather than loss kept those later, better checkpoints.
5. Keep result logs idempotent. The training script appends a row to results/summary.csv per run, so re-running the CNN experiments left duplicate rows. I report the latest run, which matches the per-seed files; writing one file per run would avoid the ambiguity.
6. Plan for disposable compute because Colab deletes everything when a session ends, so results were downloaded after each block of runs and committed to GitHub.

Limitations

- Only the first turn of each conversation and the first 256 tokens of each response are used.
- Each family pools several models of different sizes and versions, which likely hurts the weaker classes.
- The Arena data covers models from 2023 to early 2024; current models may write differently.
- Self-identification masking is a hand-written name list and may miss some mentions.
- Results come from 3 seeds on one split, so differences under about 0.01 F1 should not be over-read.

Future work

- More RQ3 evidence**:** repeat the held-out-task experiment with the BiLSTM and more seeds, and add the "other" category, to firm up the coding and math drops.
- Distinguishing features (RQ4): per-family style statistics (length, markdown use, stock phrases), plus retraining with markdown stripped to measure how much of the signal is formatting.
- Stronger models: longer inputs (512 tokens) and a pretrained transformer encoder fine-tuned on the same splits.

Repository and reproducibility

Code, results and this report: github.com/qishieeeee/llm-fingerprints. Open run.ipynb in Google Colab with a GPU runtime to rebuild the dataset and rerun every experiment. Each run folder in results/ holds per-seed metrics, confusion matrices and training curves, and results/summary.csv holds the mean ± std of every run.

References

- Chiang, W.-L., Zheng, L., Sheng, Y., et al. (2024). *Chatbot Arena: An Open Platform for Evaluating LLMs by Human Preference.* arXiv:2403.04132.
- LMArena. *arena-human-preference-55k* dataset (Apache-2.0). huggingface.co/datasets/lmarena-ai/arena-human-preference-55k.
- Kim, Y. (2014). *Convolutional Neural Networks for Sentence Classification.* EMNLP. arXiv:1408.5882.
- Hochreiter, S., & Schmidhuber, J. (1997). *Long Short-Term Memory.* Neural Computation, 9(8), 1735–1780.
- Sun, M., Yin, Y., Xu, Z., Kolter, J. Z., & Liu, Z. (2025). *Idiosyncrasies in Large Language Models.* https://arxiv.org/pdf/2502.12150